

Blood Glucose Forecasting Benchmark - Next Steps

Current Status Analysis

What's Working Well

- **Transfer Learning Implementation:** Fully functional in `run_bg_models.py` with pre-training and fine-tuning
- **Model Architecture:** Complete RNN/LSTM/GRU implementations with PyTorch
- **Data Processing:** Robust OhioT1DM data loading and preprocessing pipeline
- **Evaluation Metrics:** Comprehensive BG-specific metrics (MAE, RMSE, MARD, TIR, Clarke/Parkes EGA)
- **Results Tracking:** JSON-based experiment logging with detailed metrics
- **Base Model Interface:** Well-designed abstract base classes for model standardization

In Progress

- **Benchmark Framework:** Structure exists but needs integration with main experiment runner
- **Configuration Management:** YAML configs defined but not fully integrated
- **CLI Interface:** Basic structure exists but incomplete

Missing Critical Components

- **Experiment Runner:** Main benchmark orchestration system
- **Configuration Validation:** Schema validation for experiment configs
- **Result Analysis Tools:** Comparison and visualization utilities
- **Documentation:** API docs and usage examples
- **Testing Infrastructure:** Unit tests for reliability

Phase 1: Core Infrastructure Completion (Weeks 1-2)

Priority 1: Experiment Runner Integration

Goal: Bridge the gap between `run_bg_models.py` and the benchmark framework

Tasks:

1. Complete `benchmark/experiments/runner.py`

```
class ExperimentRunner:
    def __init__(self, config_path: str)
    def run_experiment(self) -> Dict
    def run_batch_experiments(self, config_dir: str) -> List[Dict]
```

2. Migrate Transfer Learning Logic

- Move transfer learning capabilities from `run_bg_models.py` into benchmark framework
- Create `TransferLearningExperiment` class in `benchmark/experiments/`
- Maintain backward compatibility with current `run_bg_models.py`

3. Configuration Integration

```
# Add to benchmark/configs/transfer_learning_example.yaml
experiment:
  type: "transfer_learning"
  pretrain_epochs: 50
  finetune_epochs: 25

transfer_learning:
  min_patients: 2
  validation_split: 0.2
```

Priority 2: Configuration Management System

File: `benchmark/configs/config_manager.py`

Tasks:

1. Schema Validation

```
class ConfigValidator:
    def validate_experiment_config(self, config: Dict) -> bool
    def validate_model_config(self, config: Dict) -> bool
    def get_validation_errors(self) -> List[str]
```

2. Configuration Merging

- Default config + user config merging
- Environment variable override support
- Command-line argument integration

3. Template System

- Pre-built configs for common experiments
- Dynamic config generation based on dataset/model combinations

Priority 3: CLI Enhancement

File: `benchmark/cli.py`

Tasks:

1. Command Structure

```
python -m benchmark run --config configs/my_experiment.yaml
python -m benchmark batch --config-dir configs/experiments/
python -m benchmark compare --experiments exp1,exp2,exp3
python -m benchmark validate --config configs/my_experiment.yaml
```

2. Integration with Current Workflow

```
# Maintain current functionality
python -m benchmark legacy --patients 540 544 --pretrain-epochs 5
```



Phase 2: Analysis and Reporting Tools (Weeks 3-4)

Priority 1: Result Analysis Framework

File: `benchmark/evaluation/reporting.py`

Tasks:

1. Experiment Comparison

```
class ExperimentComparator:
    def load_experiments(self, experiment_paths: List[str])
    def compare_metrics(self, metrics: List[str]) -> pd.DataFrame
    def statistical_significance_tests(self) -> Dict
    def generate_comparison_report(self) -> str
```

2. Performance Analysis

- Cross-patient performance analysis
- Model performance vs. patient characteristics
- Transfer learning effectiveness metrics
- Hyperparameter sensitivity analysis

3. Data Export

```
def export_results_to_latex_table(results: Dict) -> str
def export_to_paper_format(results: Dict, template: str) -> str
def export_leaderboard(results: List[Dict]) -> pd.DataFrame
```

Priority 2: Visualization System

File: `benchmark/utils/visualization.py`

Tasks:

1. Model Comparison Plots

```
def plot_model_comparison_radar(results: Dict) -> Figure
def plot_clarke_error_grid_comparison(results: Dict) -> Figure
def plot_performance_vs_patient_characteristics() -> Figure
```

2. Transfer Learning Analysis

```
def plot_transfer_learning_curves(history: Dict) -> Figure
def plot_patient_similarity_matrix(patients: List) -> Figure
def plot_knowledge_transfer_effectiveness() -> Figure
```

3. Interactive Dashboards

- Streamlit/Dash dashboard for result exploration
- Real-time experiment monitoring
- Model performance deep-dive tools

Phase 3: Extensibility and Robustness (Weeks 5-6)

Priority 1: Model Extension Framework

Current Gap: Easy addition of new model types

Tasks:

1. Model Registry System

```
class ModelRegistry:
    def register_model(self, name: str, model_class:
Type[BaseBGModel])
    def get_available_models(self) -> List[str]
    def create_model(self, name: str, config: Dict) -> BaseBGModel
```

2. Plugin Architecture

```
# benchmark/models/plugins/
# - transformer_models.py
# - cnn_models.py
# - ensemble_models.py
# - custom_models.py
```

3. Hyperparameter Optimization Integration

```
class HPOExperiment(BaseExperiment):
    def run_optuna_optimization(self) -> Dict
    def run_grid_search(self) -> Dict
    def run_bayesian_optimization(self) -> Dict
```

Priority 2: Dataset Extension Framework

Current Gap: Only supports OhioT1DM

Tasks:

1. Dataset Registry

```
class DatasetRegistry:
    def register_dataset(self, name: str, loader_class: Type)
    def get_available_datasets(self) -> List[str]
    def load_dataset(self, name: str, config: Dict) -> Dataset
```

2. Multi-Dataset Experiments

```
# Support for cross-dataset validation
# Dataset-specific preprocessing pipelines
# Standardized dataset interfaces
```

Priority 3: Testing Infrastructure

File: tests/

Tasks:

1. Unit Tests

```
# tests/test_models.py
# tests/test_data_loaders.py
# tests/test_metrics.py
# tests/test_experiments.py
```

2. Integration Tests

```
# tests/integration/test_full_pipeline.py
# tests/integration/test_transfer_learning.py
```

3. Performance Tests

```
# tests/performance/test_scalability.py
# tests/performance/benchmark_models.py
```

Phase 4: Advanced Features (Weeks 7-8)

Priority 1: Advanced Experiment Types

1. Cross-Validation Experiments

```
experiment:
  type: "cross_validation"
  cv_type: "time_series" # or "patient_wise", "random"
  folds: 5
```

2. Multi-Objective Optimization

```
experiment:
  type: "multi_objective"
  objectives: ["mae", "tir", "computational_cost"]
```

3. Ensemble Experiments

```
experiment:
  type: "ensemble"
  base_models: ["lstm", "gru", "transformer"]
  ensemble_method: "voting" # or "stacking", "blending"
```

Priority 2: Production Features

1. Model Versioning

```
class ModelVersioning:
    def save_model_version(self, model, version: str, metadata:
Dict)
```

```
def load_model_version(self, version: str) -> BaseBGModel
def compare_model_versions(self, versions: List[str]) -> Dict
```

2. Experiment Tracking Integration

```
# Integration with MLflow, Weights & Biases, or Neptune
class ExperimentTracker:
    def log_experiment(self, config: Dict, results: Dict)
    def log_model(self, model: BaseBGModel)
    def log_artifacts(self, artifacts: List[str])
```

3. Deployment Utilities

```
class ModelDeployment:
    def export_to_onnx(self, model: BaseBGModel) -> str
    def create_inference_api(self, model: BaseBGModel) -> FastAPI
    def benchmark_inference_time(self, model: BaseBGModel) -> Dict
```



Phase 5: Documentation and Community (Weeks 9-10)

Priority 1: API Documentation

1. Sphinx Documentation

```
docs/
├── api/           # Auto-generated API docs
├── tutorials/     # Step-by-step guides
├── examples/      # Complete example experiments
└── advanced/      # Advanced usage patterns
```

2. Jupyter Notebook Tutorials

```
# notebooks/
# - 01_getting_started.ipynb
# - 02_transfer_learning_tutorial.ipynb
# - 03_adding_custom_models.ipynb
# - 04_advanced_experiments.ipynb
```

Priority 2: Community Features

1. Leaderboard System

```
class Leaderboard:
    def submit_results(self, results: Dict, model_info: Dict)
    def get_rankings(self, metric: str = "mae") -> List[Dict]
    def generate_leaderboard_webpage(self) -> str
```

2. Model Zoo

```
# Pre-trained models for different scenarios
# benchmark/model_zoo/
# - ohiot1dm_pretrained/
# - general_bg_models/
# - specialized_models/
```

Implementation Priorities

Week 1-2 Action Items

1. **Immediate:** Complete `benchmark/experiments/runner.py` with transfer learning support
2. **High:** Implement configuration validation in `config_manager.py`
3. **High:** Create working CLI that can run current transfer learning experiments
4. **Medium:** Start migrating logic from `run_bg_models.py` to benchmark framework

Critical Dependencies

- **Config System** → **Experiment Runner** → **CLI Interface**
- **Result Analysis** → **Visualization** → **Reporting**
- **Testing** → **All Components** (parallel development)

Success Metrics

- ☐ Can run transfer learning experiment via: `python -m benchmark run --config configs/transfer_learning.yaml`
- ☐ Results are comparable to current `run_bg_models.py` output
- ☐ Configuration validation prevents invalid experiments
- ☐ Basic result comparison between experiments works
- ☐ At least 80% test coverage for core components

Key Architecture Decisions Needed

1. Backward Compatibility Strategy

- Keep `run_bg_models.py` functional during transition?
- Gradual migration vs. complete rewrite approach?

2. Result Storage Format

- Current JSON format vs. database (SQLite/PostgreSQL)?
- Experiment versioning and reproducibility requirements?

3. Model State Management

- How to handle model checkpoints during long experiments?
- Distributed training support requirements?

4. Configuration Complexity

- Simple YAML vs. programmatic configuration?
- Template system vs. full configuration language?

Success Criteria for Phase 1

By end of Phase 1, you should have:

1. **Working CLI:** `python -m benchmark run --config my_config.yaml` executes successfully
2. **Config Validation:** Invalid configurations are caught before execution
3. **Result Compatibility:** Benchmark framework produces same quality results as `run_bg_models.py`
4. **Documentation:** Clear migration guide from current workflow to benchmark framework
5. **Testing:** Core components have unit tests with >70% coverage

This roadmap will transform your current working system into a truly reusable, extensible benchmark framework while preserving all the excellent work you've already done with transfer learning and model evaluation.